

JavaScript Intermediate — Prototype Fundamentals & Prototype Chain

Section 15.1

JavaScript-এর object system বুঝতে হলে **Prototype** বুঝতেই হবে।

অনেক developer প্রথমে ভাবেন:

JavaScript-এর inheritance মানেই `class` + `extends` .

কিন্তু JavaScript-এর ভিতরের আসল mechanism হলো:

Prototype-based inheritance

`class` JavaScript-এ একটি সুন্দর syntax দেয়, কিন্তু এর নিচে prototype system-ই কাজ করে।

এই chapter-এ আমরা ধাপে ধাপে বুঝব:

```
Object
  ↓
[[Prototype]]
  ↓
__proto__
  ↓
Object.prototype
  ↓
Prototype Chain
  ↓
Property Lookup
  ↓
Inheritance
```

1. Object কী?

JavaScript-এ object হলো key-value data structure ।

```
const user = {
  name: "Shariar",
```

```
    age: 31
  };
```

এখানে:

```
user
├─ name → "Shariar"
└─ age  → 31
```

এই `user` object-এর নিজের দুটি property আছে।

```
user.name;
user.age;
```

কিন্তু interesting ব্যাপার হলো, `user` শুধু নিজের properties-ই access করতে পারে না।

```
user.toString();
```

এটাও কাজ করে।

প্রশ্ন হলো:

`toString()` তো আমরা `user` object-এর মধ্যে লিখিনি। তাহলে এটা কোথা থেকে এলো?

এখানেই Prototype-এর শুরু।

2. Prototype কী?

Prototype হলো এমন একটি object যেটা অন্য object-এর জন্য **shared properties** এবং **methods**-এর **source** হিসেবে কাজ করতে পারে।

সহজভাবে:

একটি object-এর মধ্যে কোনো property না থাকলে JavaScript তার prototype-এর মধ্যে property খুঁজতে পারে।

Example:

```
const user = {
  name: "Shariar"
};
```

```
console.log(user.toString());
```

আমরা `user`-এর মধ্যে `toString` লিখিনি।

JavaScript prototype chain অনুসরণ করে `toString()` খুঁজে পায়।

3. Prototype-based Inheritance

JavaScript-এ inheritance-এর basic idea:

```
Child Object
  ↓
Prototype
  ↓
Another Prototype
  ↓
Object.prototype
  ↓
null
```

একটি object অন্য object থেকে behavior পেতে পারে।

এটাই prototype-based inheritance-এর basic concept।

4. `[[Prototype]]` কী?

প্রতিটি normal JavaScript object-এর internal একটি reference থাকে যেটাকে specification-এ বলা হয়:

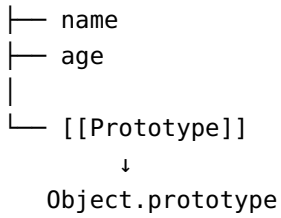
```
[[Prototype]]
```

এটা একটি **internal slot**।

এটা সাধারণ JavaScript property-এর মতো নয়।

Conceptually:

```
user
|
```



`[[Prototype]]` বলে দেয়:

"এই object-এর prototype কে?"

5. `[[Prototype]]` এবং `__proto__`

এখানে একটা খুব important distinction আছে।

`[[Prototype]]`

এটা JavaScript engine-এর internal mechanism।

`__proto__`

এটা একটি legacy accessor property, যেটার মাধ্যমে অনেক ক্ষেত্রে object's prototype access করা যায়।

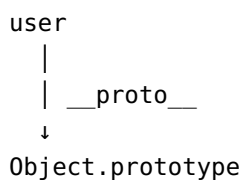
Example:

```
const user = {
  name: "Sharir"
};

console.log(user.__proto__);
```

এটা সাধারণত `Object.prototype`-কে দেখাবে।

Conceptually:



6. `prototype` এবং `__proto__` এক জিনিস নয়

এটা এই chapter-এর সবচেয়ে important distinction ।

অনেক beginner মনে করে:

```
prototype === __proto__
```

কিন্তু এগুলো একই জিনিস নয় ।

`__proto__`

একটি object-এর prototype access করার historical accessor ।

```
user.__proto__;
```

`prototype`

সাধারণত constructor function/class-এর একটি property ।

Example:

```
function User() {}  
  
console.log(User.prototype);
```

এখানে `User.prototype` একটি object ।

7. Simple Difference

এই table-টা মনে রাখো:

Term	Meaning
<code>[[Prototype]]</code>	Object-এর internal prototype reference
<code>__proto__</code>	<code>[[Prototype]]</code> access করার accessor
<code>prototype</code>	Constructor function/class-এর property
<code>Object.prototype</code>	Built-in root prototype object

সবচেয়ে important:

```
object.__proto__
```

এবং:

```
Constructor.prototype
```

দুটো different concept হলেও constructor-created objects-এর ক্ষেত্রে এদের মধ্যে relationship থাকতে পারে।

যেমন:

```
function User() {}  
  
const user = new User();  
  
console.log(user.__proto__ === User.prototype);
```

Output:

```
true
```

এটা আমরা পরের section-এ বিস্তারিতভাবে দেখব।

8. Object.prototype কী?

JavaScript-এর built-in `Object` constructor-এর একটি prototype object আছে:

```
Object.prototype
```

এটা খুব গুরুত্বপূর্ণ।

Example:

```
console.log(Object.prototype);
```

এখানে সাধারণ object-এর অনেক common methods পাওয়া যায়:

```
toString
hasOwnProperty
isPrototypeOf
propertyIsEnumerable
valueOf
...
```

তাই:

```
const user = {
  name: "Sharir"
};

user.toString();
```

কাজ করে।

কারণ:

```
user
  ↓
Object.prototype
  ↓
toString()
```

9. `Object.prototype` হলো **Prototype Chain**-এর গুরুত্বপূর্ণ অংশ

সাধারণ object-এর জন্য chain সাধারণত:

```
user
  ↓
Object.prototype
  ↓
null
```

এখানে:

- `user` → আমাদের object

- `Object.prototype` → built-in prototype
- `null` → chain-এর শেষ

10. Prototype Chain কী?

যখন JavaScript কোনো property current object-এ খুঁজে পায় না, তখন prototype-এর মধ্যে খোঁজে।

প্রয়োজনে prototype-এর prototype-এ যায়।

এই sequence-টাই:

Prototype Chain

Example:

```
const user = {  
  name: "Sharlar"  
};  
  
console.log(user.toString);
```

JavaScript conceptually:

```
1. user-এর মধ্যে toString আছে?  
   ↓  
   না  
  
2. user-এর [[Prototype]]-এ আছে?  
   ↓  
Object.prototype  
   ↓  
   হ্যাঁ
```

তাই `toString` পাওয়া যায়।

11. Property Lookup কীভাবে কাজ করে?

ধরো:


```
const user = {  
  name: "Sharir"  
};  
  
console.log(user.name);
```

JavaScript প্রথমে:

```
user  
↓  
name?
```

পেয়ে যায়।

তাই `search` এখানেই শেষ।

কিন্তু:

```
console.log(user.toString());
```

এর ক্ষেত্রে:

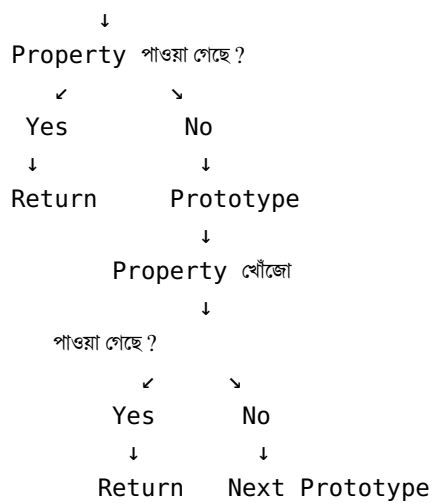
```
user  
↓  
toString?  
↓  
না  
  
Object.prototype  
↓  
toString?  
↓  
হ্যাঁ
```

তাই `Object.prototype.toString` পাওয়া যায়।

12. Property Lookup Mental Model

সবসময় এই pattern মনে রাখো:

Current Object



শেষে:

null

পৌঁছে গেলে আর খোঁজার জায়গা নেই।

তখন result সাধারণত:

undefined

13. Example: Prototype Chain

```
const user = {  
  name: "Sharlar"  
};  
  
console.log(user.name);  
console.log(user.toString);  
console.log(user.xyz);
```

Results:

```
"Shariar"  
function toString() { ... }  
undefined
```

কেন?

name

নিজের property।

toString

Prototype থেকে inherited।

xyz

কোথাও পাওয়া যায়নি।

14. Own Property কী?

যে property সরাসরি object-এর নিজের, সেটা **own property**।

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

এখানে:

```
name → own  
age → own
```

Check:

```
console.log(Object.hasOwn(user, "name"));
```

Output:

true

15. Inherited Property কী?

যে property object নিজে রাখে না, prototype থেকে পায়, সেটা inherited property।

```
const user = {  
  name: "Sharir"  
};  
  
console.log(user.toString);
```

toString:

user-এর own property নয়
↓
Object.prototype থেকে inherited

তাই:

```
Object.hasOwn(user, "toString");
```

হবে:

false

কিন্তু:

```
"user" in user
```

এর মতো `in` operator inherited properties-ও check করে।

Example:

```
console.log("toString" in user);
```

Output:

true

16. Own vs Inherited

Property	Own?	Inherited?
user.name	✓	✗
user.age	✓	✗
user.toString	✗	✓
user.hasOwnProperty	✗	✓

17. Shadowing + Prototype

Prototype chain-এর সাথে **shadowing** খুব important ।

ধরো:

```
const user = {  
  toString: "My own value"  
};  
  
console.log(user.toString);
```

Output:

My own value

এখন JavaScript `Object.prototype.toString` পর্যন্ত যাবে না ।

কারণ current object-এই `toString` পাওয়া গেছে ।

এটাকে বলা যায় prototype-level shadowing ।

Mental model:

```
user
└─ toString = "My own value"
    ↓
  STOP SEARCH

Object.prototype
└─ toString()
```

উপরের property নিচের inherited property-কে hide করছে।

18. Prototype Chain Example with Multiple Levels

আমরা manually prototype chain তৈরি করতে পারি।

```
const animal = {
  eats: true
};

const dog = Object.create(animal);

dog.barks = true;

console.log(dog.barks);
console.log(dog.eats);
```

Output:

```
true
true
```

কেন?

```
dog
├─ barks = true
└─ [[Prototype]]
    ↓
  animal
    └─ eats = true
```

`dog` নিজের মধ্যে `eats` না পেলেও `animal`-এ পেয়ে গেছে।

19. আরও এক Level

```
const livingThing = {
  alive: true
};

const animal = Object.create(livingThing);

animal.eats = true;

const dog = Object.create(animal);

dog.barks = true;
```

Chain:

```
dog
↓
animal
↓
livingThing
↓
Object.prototype
↓
null
```

এখন:

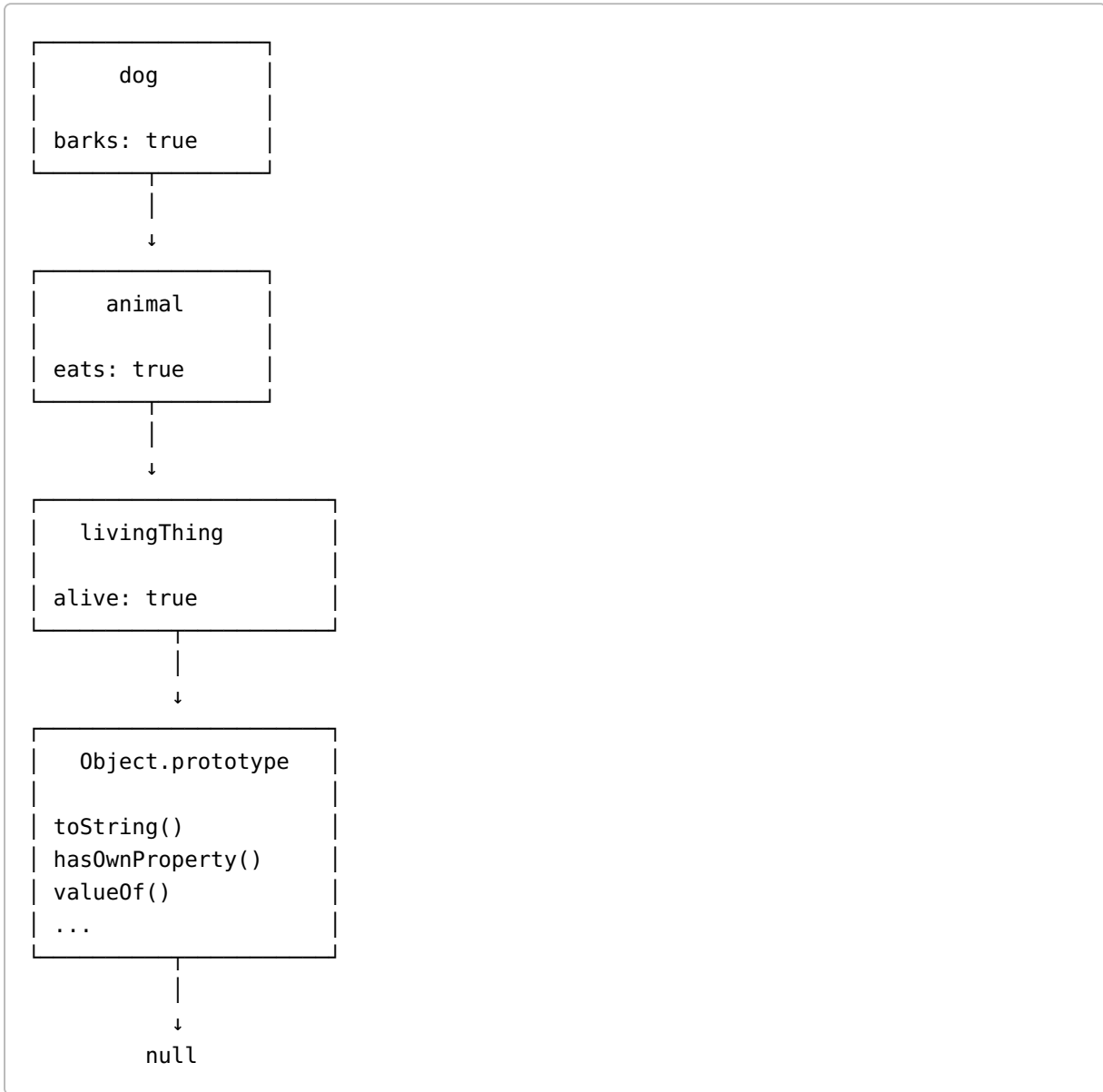
```
console.log(dog.barks);
console.log(dog.eats);
console.log(dog.alive);
```

Output:

```
true
true
true
```

প্রতিটি property আলাদা level থেকে পাওয়া যাচ্ছে।

20. Prototype Chain Diagram



21. `Object.getPrototypeOf()`

Prototype inspect করার modern এবং recommended way হলো:


```
Object.getPrototypeOf(object)
```

Example:

```
const user = {  
  name: "Shariar"  
};  
  
const prototype = Object.getPrototypeOf(user);  
  
console.log(prototype === Object.prototype);
```

Output:

```
true
```

22. `__proto__` বনাম `Object.getPrototypeOf()`

দুটো দিয়ে prototype দেখা যেতে পারে:

```
user.__proto__;
```

এবং:

```
Object.getPrototypeOf(user);
```

Modern code-এ সাধারণত prefer করা হয়:

```
Object.getPrototypeOf(user);
```

কারণ এটা standard API এবং intent অনেক clearer ।

23. `Object.setPrototypeOf()`

Prototype পরিবর্তন করা যায়:

```
const animal = {
  eats: true
};

const dog = {
  barks: true
};

Object.setPrototypeOf(dog, animal);

console.log(dog.eats);
```

Output:

```
true
```

এখন:

```
dog
↓
animal
```

24. `Object.setPrototypeOf()` কেন সাবধানে ব্যবহার করতে হয়?

Existing object-এর prototype বারবার পরিবর্তন করা performance-এর জন্য খারাপ হতে পারে এবং code-এর behavior confusing করতে পারে।

তাই সাধারণ application code-এ prototype dynamically change করার বদলে:

```
Object.create(proto)
```

বা proper constructor/class design ব্যবহার করা বেশি appropriate।

25. `Object.create()` দিয়ে **Prototype Relationship**

```
const animal = {  
  eats: true  
};  
  
const dog = Object.create(animal);
```

এখানে `dog`-এর:

```
[[Prototype]]
```

হলো:

```
animal
```

Check:

```
console.log(Object.getPrototypeOf(dog) === animal);
```

Output:

```
true
```

26. **Prototype Property** কি সব **Object**-এর থাকে?

এখানে সাবধান।

```
const user = {  
  name: "Sharlar"  
};  
  
console.log(user.prototype);
```

Output:

```
undefined
```

সাধারণ object-এর নিজের `prototype` property থাকে না।

`prototype` property সাধারণত constructor functions-এর সাথে associated।

Example:

```
function User() {}  
  
console.log(User.prototype);
```

এখানে পাওয়া যাবে।

27. Function-এর Prototype

Function হলো object-এর একটি special type।

```
function User() {}
```

এই function-এর একটি property থাকে:

```
User.prototype
```

এটা নতুন instance-গুলোর prototype হিসেবে ব্যবহৃত হয় যখন:

```
new User()
```

করা হয়।

Example:

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");
```

```
console.log(
  Object.getPrototypeOf(user) === User.prototype
);
```

Output:

```
true
```

এটা পরের section-এ **new** এবং constructor function-এর সময় বিস্তারিত করব।

28. Class-এর ক্ষেত্রেও **Prototype** আছে

```
class User {
  greet() {
    console.log("Hello");
  }
}
```

এখন:

```
console.log(User.prototype);
```

এবং:

```
const user = new User();

console.log(
  Object.getPrototypeOf(user) === User.prototype
);
```

Output:

```
true
```

অর্থাৎ class ব্যবহার করলেও prototype system কাজ করছে।

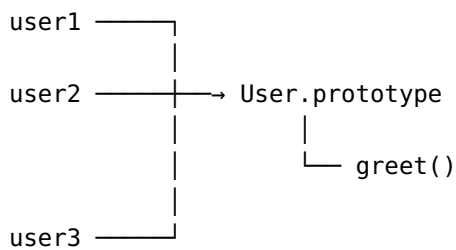
29. Class Method কোথায় থাকে?

ধরো:

```
class User {  
  greet() {  
    console.log("Hello");  
  }  
}  
  
const user1 = new User();  
const user2 = new User();
```

দুইটা object-এর মধ্যে আলাদা আলাদা `greet` function copy থাকার দরকার নেই।

বরং:



এভাবে method shared থাকে।

এটাই prototype-এর বড় advantage।

30. কেন Prototype ব্যবহার করা হয়?

ধরো 1000টা user object আছে।

যদি প্রতিটা object-এর মধ্যে একই method copy করা হয়:

```
const user1 = {  
  name: "A",  
  greet() {}  
};  
  
const user2 = {
```

```

    name: "B",
    greet() {}
};

const user3 = {
  name: "C",
  greet() {}
};

```

তাহলে conceptually অনেক duplicate function তৈরি হতে পারে।

Prototype-based approach:

```

user1 ┌
user2 ┌
user3 ┌ → shared greet()
user4 ┌
user5 ┌

```

একটি shared method ব্যবহার করা যায়।

31. Inheritance Example

```

const animal = {
  eat() {
    console.log("Eating");
  }
};

const dog = Object.create(animal);

dog.bark = function () {
  console.log("Barking");
};

dog.eat();
dog.bark();

```

Output:

Eating
Barking

bark():

dog-এর own method

eat():

animal থেকে inherited method

32. Prototype Chain-এ Property পাওয়া গেলে Search Stop

```
const parent = {  
  value: 10  
};  
  
const child = Object.create(parent);  
  
child.value = 20;  
  
console.log(child.value);
```

Output:

20

JavaScript:

```
child.value  
  ↓  
child-এ পাওয়া গেছে  
  ↓  
STOP
```

parent.value check করার দরকার নেই।

33. Delete করলে কী হয়?

```
const parent = {  
  value: 10  
};  
  
const child = Object.create(parent);  
  
child.value = 20;  
  
delete child.value;  
  
console.log(child.value);
```

Output:

10

কেন?

আপে:

```
child  
└─ value = 20
```

delete করার পর:

```
child  
└─ no value  
  
    ↓ prototype  
  
parent  
└─ value = 10
```

তাই inherited value পাওয়া যায়।

34. Prototype Property Mutation

Inherited object mutable হলে child সেই change দেখতে পারে।

```
const parent = {  
  role: "user"  
};  
  
const child = Object.create(parent);  
  
console.log(child.role);
```

Output:

user

পরে:

```
parent.role = "admin";  
  
console.log(child.role);
```

Output:

admin

কারণ `child` নিজের copy রাখেনি।

সে prototype-এর property lookup করছে।

35. Own Property তৈরি করলে

```
const parent = {  
  role: "user"  
};  
  
const child = Object.create(parent);  
  
child.role = "admin";  
  
console.log(child.role);
```

Output:

```
admin
```

এখন:

```
child
└─ role = "admin"

parent
└─ role = "user"
```

Child-এর own property prototype-এর একই নামের property-কে shadow করেছে।

36. `in` Operator

`in` operator object এবং তার prototype chain—দুটোতেই search করে।

```
const user = {
  name: "Shariar"
};

console.log("name" in user);
console.log("toString" in user);
console.log("xyz" in user);
```

Output:

```
true
true
false
```

কারণ:

```
name → own property
toString → inherited property
xyz → nowhere
```

37. Object.hasOwn()

শুধু own property check করতে:

```
const user = {  
  name: "Sharlar"  
};  
  
console.log(Object.hasOwn(user, "name"));  
console.log(Object.hasOwn(user, "toString"));
```

Output:

```
true  
false
```

এটা prototype chain-এ search করে না।

38. hasOwnProperty() এর সাথে সম্পর্ক

পুরোনো/common approach:

```
user.hasOwnProperty("name");
```

Modern approach:

```
Object.hasOwn(user, "name");
```

`Object.hasOwn()` সম্পর্কে বিস্তারিত আমরা Section 15.4-এ করব।

এই section-এ শুধু মনে রাখো:

```
in  
↓  
own + inherited  
  
Object.hasOwn()
```

↓
own only

39. Object.prototype থেকে Common Methods

এই methods সাধারণ objects-এর prototype chain-এর মাধ্যমে পাওয়া যায়:

```
const user = {  
  name: "Shariar"  
};  
  
user.toString();  
user.valueOf();  
user.hasOwnProperty("name");
```

এগুলো আমরা object-এর মধ্যে manually declare করিনি।

তাই:

```
user  
↓  
Object.prototype  
↓  
common object methods
```

40. Object.create(null)

একটি object-এর prototype না থাকলেও object তৈরি করা যায়।

```
const dictionary = Object.create(null);
```

এখানে:

```
Object.getPrototypeOf(dictionary);
```

Output:

```
null
```

অর্থাৎ:

```
dictionary
  ↓
  null
```

কোনো `Object.prototype` chain নেই।

41. কেন `Object.create(null)` Useful?

এটা pure dictionary/map-like object হিসেবে ব্যবহার করা যায় যেখানে inherited properties দরকার নেই।

```
const dictionary = Object.create(null);

dictionary.name = "Shariar";
dictionary.age = 31;
```

এখানে:

```
dictionary.toString
```

এর মতো inherited method থাকবে না।

তাই special dictionary use cases-এ এটা useful হতে পারে।

42. Prototype Chain-এর শেষ কেন `null`?

Normal object:

```
object
  ↓
Object.prototype
  ↓
null
```

`Object.prototype` নিজেও একটি object।

কিন্তু তার prototype:

```
Object.getPrototypeOf(Object.prototype);
```

হলো:

```
null
```

অর্থাৎ:

```
Object.prototype
  ↓
  null
```

এটাই chain-এর শেষ।

43. Full Example

```
const animal = {
  eats: true
};

const dog = Object.create(animal);

dog.name = "Buddy";

console.log(dog.name);
console.log(dog.eats);
console.log(dog.toString);
```

Search:

`dog.name`

```
dog
  ↓
name found
```

`dog.eats`

```
dog
↓
eats not found
↓
animal
↓
eats found
```

`dog.toString`

```
dog
↓
animal
↓
Object.prototype
↓
toString found
```

Full chain:

```
dog
↓
animal
↓
Object.prototype
↓
null
```

44. Prototype vs Class — Important Perspective

JavaScript-এ:

```
class User {
  greet() {}
}
```

দেখে মনে হতে পারে:


```
User
↓
class inheritance system
```

কিন্তু বাস্তবে class-এর methods prototype-এর মাধ্যমে shared হয়।

Conceptually:

```
User instance
↓
User.prototype
↓
Object.prototype
↓
null
```

তাই:

Classes did not replace prototypes. Classes provide a cleaner syntax over JavaScript's prototype-based object model.

45. Prototype Chain vs Scope Chain

দুটোকে গুলিয়ে ফেলবে না।

Scope Chain

Variable খোঁজার জন্য:

```
Current Scope
↓
Outer Scope
↓
Global Scope
```

Prototype Chain

Object property খোঁজার জন্য:

```

Object
↓
Prototype
↓
Prototype's Prototype
↓
Object.prototype
↓
null

```

Difference:

Scope Chain	Prototype Chain
Variables খোঁজে	Object properties খোঁজে
Lexical scope-এর উপর ভিত্তি করে	Object prototype-এর উপর ভিত্তি করে
Function/block scope involved	Objects involved
Closure-এর সাথে related	Inheritance-এর সাথে related

46. Prototype Chain + Scope Chain একসাথে

একটা function-এর মধ্যে object থাকলে দুটো system-ই কাজ করতে পারে।

```

const role = "global";

function test() {
  const user = {
    name: "Shariar"
  };

  console.log(role);
  console.log(user.toString());
}

```

`role` খুঁজতে:

Scope Chain

ব্যবহার হবে।

`user.toString` খুঁজতে:

Prototype Chain

ব্যবহার হবে।

47. Common Mistakes

Mistake 1 — `prototype` এবং `__proto__` একই মনে করা

ভুল।

```
__proto__  
→ object's prototype access  
  
prototype  
→ constructor function/class-এর property
```

Mistake 2 — সব **object**-এর `.prototype` থাকে

ভুল।

```
const user = {};  
  
console.log(user.prototype);
```

Output:

undefined

Mistake 3 — `__proto__` এবং `[[Prototype]]` completely different

এটাও পুরোপুরি ঠিক নয়।

`[[Prototype]]` হলো internal slot।

`__proto__` হলো সেই prototype-এর জন্য একটি accessor mechanism ।

Mistake 4 — Inherited property মানেই object-এর নিজের property

না।

```
const user = {};  
  
Object.hasOwn(user, "toString");
```

হবে:

false

কিন্তু:

```
"toString" in user
```

হবে:

true

Mistake 5 — Prototype মানেই parent class

সবসময় class-এর ধারণায় prototype বোঝার চেষ্টা করলে confusion হবে।

Prototype হলো object-to-object delegation/inheritance mechanism ।

48. Interview Questions

Basic

1. Prototype কী?
2. JavaScript prototype-based language বলতে কী বোঝায়?
3. `[[Prototype]]` কী?
4. `__proto__` কী?
5. `Object.prototype` কী?

6. Prototype chain কী?
7. Property lookup কীভাবে কাজ করে?
8. Own property কী?
9. Inherited property কী?
10. `in` operator prototype chain check করে কি?

Intermediate

1. `prototype` এবং `__proto__` -এর difference কী?
2. `Object.getPrototypeOf()` কী করে?
3. `Object.setPrototypeOf()` কেন সাবধানে ব্যবহার করা উচিত?
4. `Object.create()` কীভাবে prototype relationship তৈরি করে?
5. `Object.prototype` chain-এর কোথায় থাকে?
6. `Object.prototype` -এর prototype কী?
7. Prototype property shadowing কী?
8. `Object.create(null)` কী?
9. Scope chain এবং prototype chain-এর difference কী?
10. JavaScript class কি সত্যিই prototype system replace করে?

49. Output Prediction Practice

Question 1

```
const parent = {  
  value: 10  
};  
  
const child = Object.create(parent);  
  
console.log(child.value);
```

Output কী?

Question 2

```
const parent = {  
  value: 10  
};  
  
const child = Object.create(parent);  
  
child.value = 20;
```

```
console.log(child.value);  
console.log(parent.value);
```

Output କି?

Question 3

```
const parent = {  
  value: 10  
};  
  
const child = Object.create(parent);  
  
child.value = 20;  
  
delete child.value;  
  
console.log(child.value);
```

Output କି?

Question 4

```
const user = {  
  name: "Shariar"  
};  
  
console.log(Object.hasOwn(user, "name"));  
console.log(Object.hasOwn(user, "toString"));  
console.log("toString" in user);
```

Output କି?

Question 5

```
const animal = {  
  eats: true  
};
```

```
const dog = Object.create(animal);  
  
console.log(Object.getPrototypeOf(dog) === animal);
```

Output কী?

50. Practice Tasks

Task 1 — Prototype Chain

তিনটি object তৈরি করো:

```
livingThing  
  ↓  
animal  
  ↓  
dog
```

Properties:

```
livingThing → alive  
animal      → eats  
dog         → barks
```

তারপর `dog` থেকে তিনটি property-ই access করো।

Task 2 — Shadowing

একটি parent object-এ:

```
role: "user"
```

দাও।

Child object-এ একই:

```
role: "admin"
```

দাও।

তারপর `parent` এবং `child`-এর `value compare` করো।

Task 3 — Own vs Inherited

একটি `object` তৈরি করো যার `prototype`-এ:

```
country: "Bangladesh"
```

আছে এবং `object`-এর নিজের:

```
name: "Sharlar"
```

আছে।

তারপর:

```
Object.hasOwnProperty()
```

দিয়ে দুইটা `property check` করো।

Task 4 — Prototype Investigation

এই `object`-এর `prototype` বের করো:

```
const user = {  
  name: "Sharlar"  
};
```

ব্যবহার করো:

```
Object.getPrototypeOf()
```

তারপর দেখো:

```
Object.getPrototypeOf(user) === Object.prototype
```


কেন `true` হচ্ছে সেটা নিজের ভাষায় explain করো।

51. Quick Reference

`[[Prototype]]`

↓

Object-এর internal prototype reference

`__proto__`

↓

Prototype access করার legacy accessor

`Object.prototype`

↓

সাধারণ object-এর গুরুত্বপূর্ণ built-in prototype

Prototype Chain

↓

Object

↓

Prototype

↓

Next Prototype

↓

`Object.prototype`

↓

null

`Object.hasOwn(obj, key)`

↓

Own property only

`key in obj`

↓

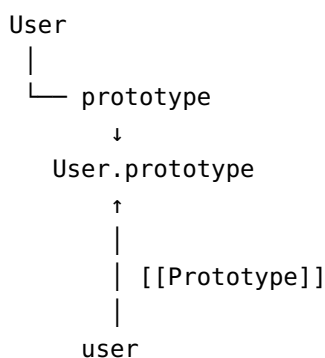
Own + inherited property

52. The Most Important Relationship

এই relationship-টা খুব ভালোভাবে বুঝে রাখো:

```
function User() {}  
  
const user = new User();
```

Conceptually:



অর্থাৎ:

```
Object.getPrototypeOf(user) === User.prototype
```

হবে:

```
true
```

এটাই পরের chapter-এর foundation ।

53. Final Mental Model

JavaScript object-এর property access-এর সময় প্রথমে:

```
নিজের object  
↓  
property আছে?
```

না থাকলে:

```
[[Prototype]]  
↓  
property আছে?
```

না থাকলে:

```
prototype-এর prototype  
↓  
property আছে?
```

এভাবে:

```
Object  
↓  
Prototype  
↓  
Prototype  
↓  
Object.prototype  
↓  
null
```

পর্যন্ত যায়।

এই পুরো process-টাই:

Prototype Chain

আর JavaScript-এর inheritance মূলত এই mechanism-এর উপর দাঁড়িয়ে আছে।

Mastery Checklist

- [] Prototype কী বুঝি
- [] [[Prototype]] বুঝি
- [] __proto__ কী বুঝি
- [] prototype কী বুঝি
- [] prototype vs __proto__ difference বলতে পারি
- [] Object.prototype বুঝি
- [] Prototype chain আঁকতে পারি

- [] Property lookup explain করতে পারি
- [] Own property বনাম inherited property বুঝি
- [] `Object.getPrototypeOf()` ব্যবহার করতে পারি
- [] `Object.setPrototypeOf()` সম্পর্কে জানি
- [] `Object.create()` দিয়ে prototype relationship বুঝি
- [] Shadowing + prototype বুঝি
- [] `in` বনাম `Object.hasOwnProperty()` basic difference বুঝি
- [] `Object.create(null)` বুঝি
- [] Scope chain বনাম prototype chain আলাদা করতে পারি
- [] বুঝি যে JavaScript classes-এর নিচেও prototype system কাজ করে